

Syntax as the growth of transparent semantic representations

An executable research workbench for Dynamic Syntax

Classical semantics / Constructive semantics / TTR

8 October 2026

ds-workbench.vercel.app

Built with AI assistance

Development tools and model access

Development

Astra with **Codex**

Fable 5.1 with **Claude Code**

Models in the application

OpenRouter provides access to all models used in the app.

The research question

Can we broaden coverage while keeping derivations inspectable?

For a linguist

- Which word licensed each operation?
- Which requirements are still open?
- Where did this meaning come from?

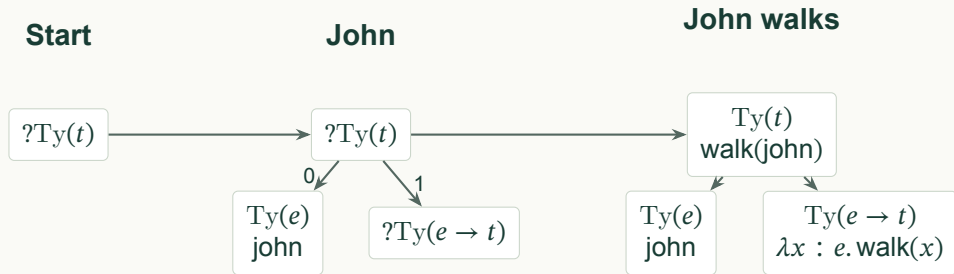
For an engineering system

- Use dictionaries and corpora.
- Let models propose useful analyses.
- Retain failures and alternatives.

The DS engine must build the derivation. A model suggestion is an assumption to examine.

One word changes the state

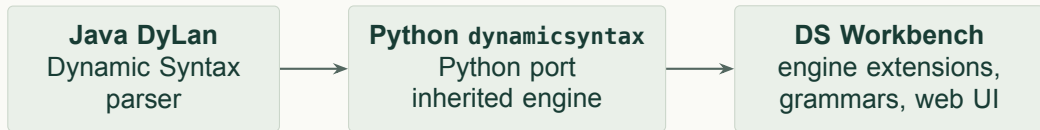
A tree, a pointer, and requirements



Schematic Classical states after local completion; intermediate pointer movements are omitted.

What the project is built on

Software foundation



- dynamicsyntax: public Python API, grammar discovery, results.
- dylan: most parser, tree, action and formula implementation.
- A DS parser with substantial TTR machinery is the foundation.

What the workbench adds

	Inherited foundation	Extensions in this workbench
Parsing	Incrementality, action language, alternatives, context DAG	Recovery, limits, diagnostics, replay and state isolation
Semantics	TTR records and composition	Native Classical/Constructive terms; TTR identity corrections
Grammar	Bundled resources and induction infrastructure	English/Greek fragments; typed templates; supplemental relatives
Application	Existing interfaces and visualisation work	Focused browser views, live rule replay, research library and community source

The contribution includes changes to the engine and grammars, as well as the interface.

Shared DS machinery, three semantic implementations

		Classical	Constructive	TTR
Representations		Predicates, λ , ε/τ choice terms	Typed witnesses, Σ/Π , projections	Record types, fields and dependencies
Composition		Typed application and reduction	Application, subtyping and witness closure	Record functions, paths and record merge
Current assistance	assis-	Native English / Standard Greek	Native English / Standard Greek	No construction or vocabulary compiler in this interface

Selecting a backend selects its grammar and semantic operations. It does not reformat a completed TTR parse.

The same sentence, different semantic objects

A man knows you.

Classical

$$\text{know}(\exists x : e. \text{man}(x), \text{you})$$

Constructive: witness supplied to the predicate

$$\Sigma p : (\Sigma x : \text{man}. \top). \text{know}(\pi_1(p), \text{you})$$

Constructive: simplified interpretation

$$\Sigma x : \text{man}. \text{know}(x, \text{you})$$

An interpretation type is not a constructed proof that the assertion is true.

TTR keeps participant and event dependencies explicit

x_0	$: e$
$p_0 = \text{man}(x_0)$	$: t$
$x_1 = \text{you}$	$: e$
$e_0 = \text{know}$	$: es$
$p_2 = \text{pres}(e_0)$	$: t$
$p_5 = \text{subj}(e_0, x_0)$	$: t$
$p_4 = \text{obj}(e_0, x_1)$	$: t$
$\text{head} = e_0$	$: es$

- Subject and object retain their own dependencies.
- Freshening must preserve constants and field identity.
- Native grammar extensions do not automatically extend TTR.

Readable presentation of the recorded TTR analysis; field equality here denotes a manifest field.

Coverage is a set of executable fragments

Native English

- Names, determiners and nominal constructions.
- Verb frames, copulas, PP arguments and modifiers.
- Relatives and finite clause relations.
- Limited questions, reflexives and dialogue repairs.

Native Standard Greek

- Case-sensitive names and nominal constructions.
- Clitics, clusters, verb frames and selected word orders.
- PP goals, comparisons and finite clause relations.
- Other varieties have smaller reviewed fragments.

Coverage is defined by supported constructions, not by the language label.

A literature map now guides implementation

18 categories / 91 tracked topics / source-linked obligations

Families in the map	Examples of theoretical coverage
Growth and predication	Lexical programs, nominals, scope, valency and copulas
Dependencies	Flexible order, relatives, resumption, extraction and modification
Reference and dialogue	Coordination, binding, clitics, ellipsis, repair and grounding
Clauses and variation	Auxiliaries, tense/voice, clause linkage, agreement and historical change
Computation	Learning, search, generation and semantic architectures

An analysis in the literature is a starting point for a program and semantic tests. It is not automatically implemented here.

A searchable library connects papers to implementation

53 selected works / 30 source versions / 8 analysis cards

Find and inspect

- Open the app's **DS Library** tab.
- Search text; filter topic and semantic framework.
- See review scope and publication links.

Trace an analysis

1. Work and source version.
2. Bounded claim and exact passages.
3. DS mechanisms and semantic obligations.
4. Backend-specific code and tests.

Most readings are partial. A bibliographic record never establishes implemented coverage.

PP arguments and modifiers do different jobs

Sentence	Implemented treatment
John relies on Mary .	Selected verb argument: an <i>on</i> -complement is required.
John gives a book to Mary .	Recipient argument in the dative-to frame.
John walks in a park .	Modifier grows through LINK; its own NP is parsed.
A man is in a park .	Copular locative predicate.

`in_location(place, walk(john))`

Schematic modifier meaning. The fragment does not supply a spatial or event-inference model.

The same LINK mechanism can compose different meanings

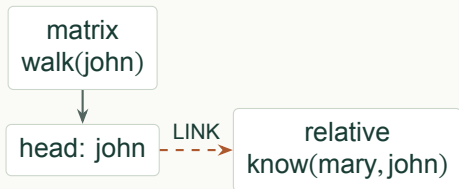
Construction	Example	Semantic obligation
Restrictive relative	A man who Mary knows walks.	Refine the nominal restrictor before quantification.
Nonrestrictive relative	John, who Mary knows, walks.	Preserve John; add a supplemental assertion.
PP modifier	John walks in a park.	Attach the locative content at the intended scope.

Specify what is copied, where requirements close, and how the linked content is evaluated.

Cann, Kempson & Marten (2005), ch.3; Howes & Gibson (2021), §§2.1, 2.5.1.

From the source analysis to an executable relative

John, who Mary knows, walks. No LLM needed.



Schematic: functor branches and intermediate states omitted.

1. Comma opens a LINKed proposition.
2. *Who* copies the head to an unfixed node.
3. MERGE fixes the copy as the object of *knows*.
4. Close the relative; return and compose.

$\text{walk}(\text{john}) \wedge \text{know}(\text{mary}, \text{john})$

Cann, Kempson & Marten, §3.1, rules (3.6), (3.9), (3.13); inspected draft PDF pp.84–90.

Test the relative's meaning and its boundaries

Implemented in both native English backends

- Named subject or direct-object hosts.
- Subject or direct-object gaps.
- Local negation, adverbs and indefinite witnesses.

Still outside this claim

- Quantified or complex definite hosts.
- Nonlocal gaps and embedded projection.
- Greek relatives and TTR parity.

56 focused tests; 8 public browser checks: six complete derivations and two expected incomplete cases.

7 October: authored construction checks, not an estimate of unseen-text accuracy.

Greek clitics: order, case and tree growth

της τον έδωσε. “S/he gave him to her.”

- Genitive recipient and accusative theme.
- Unfixed structure, MERGE and substitution.
- Ditransitive frame preserves three participants.

Readable role structure

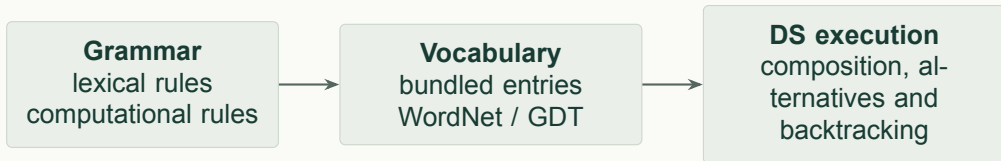
give(pro, him, her)

subject theme recipient

Predicate renamed for readability; pronoun reference is assumed.

PCC control: *του με έδωσε* fails in the modeled SMG clitic reading, in both native backends.

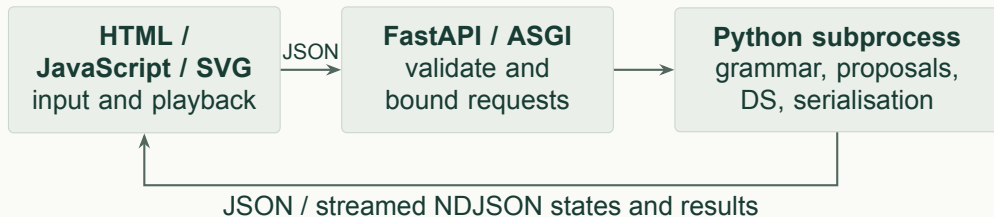
What already works without an LLM



- Tree growth, meaning composition, diagnostics and playback.
- Sentence, paragraph and supported dialogue flows.
- Corpus/dictionary investigation and exports.

A model key is optional. Dictionaries still supply candidates, not guaranteed contextual meanings.

From the browser to an isolated DS worker



- Process isolation protects mutable parser and metavariable state.
- The browser renders recorded states; it does not run the parser.

The model and the engine have distinct responsibilities

	Selected chat model	DS engine and compiler
Vocabulary	Propose lemma, sense, morphology and argument frame	Validate fields and instantiate an allowed lexical template
Constructions	Select a declared clause relation; propose a connective use	Instantiate a reviewed construction and compose its contents
Failure	Revise a hypothesis using actual DS feedback	Retry within limits; retain incomplete results

Model output contains structured hypotheses. It does not contain executable code or a finished replacement tree.

Two switches choose what the next parse may use

Use analysis LLM / Use Jev

LLM	Jev	Enabled route
OFF	OFF	DS with the selected original, bundled, dictionary or corpus vocabulary.
ON	OFF	Bounded lexical and construction assistance when needed.
OFF	ON	Jev word-meaning preferences; no generative fallback.
ON	ON	Jev preferences plus lexical-model fallback; no construction assistance in this combined mode.

Connect / models sets up access. The switches authorize use.
This result reports the requests actually made.

A construction family replaces one-off conjunction fixes

Surface word → declared relation → shared DS program

Family	English examples	Greek examples
Cause	because; causal since	επειδή, διότι; causal αφού
Time	when, while, before, after, until	όταν, όποτε, ωστόσο
Condition	if, unless	αν, εάν
Concession / contrast	although, though, whereas	παρότι, μολονότι; readings of ενώ

Postposed clauses grow through LINK. Fronted clauses require an explicit comma and a main clause. Ambiguous words retain alternatives.

A recorded example of model assistance

John walks provided Mary walks.

Before assistance

The available lexical programs do not license this connective use.

Model proposal

```
surface: provided  
relation: condition
```

Actual DS result

```
condition(walk(john),  
          walk(mary))
```

The UI exposes the compiled template and the relation on the accepted DS path.

Recorded 4 October checks: *provided* and Greek *καθότι* completed in both native backends.

Jev has a different, visible role

John lends a book to Mary.

DS without Jev preferences

“bestow a quality on”

A complete but contextually poor lexical-sense choice.

DS with Jev preferences

“give temporarily”

DS replay accepts a preferred sense after more context arrives.

- Two actual runs use identical lexical programs.
- Jev ranks senses from the observed prefix; alternatives survive.
- Its pinned model is separate from the selected chat model.

Recorded demonstration, not an accuracy or speed benchmark. The independent search-order control remains gated by calibration.

Corpora and dictionaries supply evidence

Lexical resources

- English: reviewed additions; optional WordNet index.
- Greek: GDT training export.
- 2,021 Greek forms; 2,330 analyses.

Optional live Greek evidence

- Svarna examples and corpus context.
- Triantafyllidis headwords and excerpts.
- Observations and citations accompany lexical proposals.

An attested occurrence is evidence for an analysis. It is not an exhaustive valency specification or automatic grammar training.

Sentences, paragraphs and dialogue

Input	State handling	Important boundary
Sentence	Incremental parse and alternative complete analyses	Completion is relative to the selected grammar
Paragraph	Completed sentences retain DS context	Failures count and become explicit context gaps
Dialogue	A new speaker can continue the same tree; supported repairs preserve history	General ellipsis, grounding and discourse inference remain incomplete

A later successful sentence does not turn an incomplete paragraph into a complete interpretation.

A workspace organised around the growing tree

Parse | Corpus & dictionary | Greek lab | DS Library

Input and semantic system / visible LLM and Jev switches

Tree and playback
Meaning directly below the tree

Node inspector
and active rule

Evidence & lexical analyses entries grouped by word

Settings open in a drawer. Tabs preserve the parse. Manual zoom persists; Auto-fit resumes following the tree.

Computational rules appear while the tree grows

Rules is the default playback level

See the active program

- Lexical and computational rules in execution order.
- Introduction and anticipation; completion and elimination.
- Pointer movement and changed decorations.

Choose the level of detail

- **Words:** states at word boundaries.
- **Rules:** each lexical/computational action.
- **Operations:** make/go/put inside a rule, where recorded.

Backtracking restores a prior state. **Show final result** returns to the endpoint.

Evidence for coverage: keep the samples separate

Check	English	Greek
Unseen-source baseline (1 Oct.)	0/6 paragraphs	0/6 passages
Hand-written assisted development set	6/9 sentences	5/9 sentences
New clause paragraphs	3/3 sentences	3/3 sentences
Live new-connective examples	provided completed	καθότι completed

- Results above were checked in both native backends.
- The frozen baseline predates the latest extensions.
- These small samples are not a representative accuracy estimate.

What “complete” does—and does not—establish

It establishes

- A path through the implemented DS programs.
- Satisfied tree requirements and available type checks.
- A composed meaning under displayed assumptions.

It does not establish

- Correct sense, reference or attachment.
- Independent grammaticality or truth.
- Full temporal, causal or counterfactual reasoning.

Clause relations are distinct, typed, opaque predicates. That is useful structure, but still a limited semantics.

A resource other DS researchers can use and extend

Starting point	What is available	A useful contribution
Explore / teach	Browser, traces, examples and slides	An exercise or sourced counterexample
Reproduce	Local install, Python API, exported results	Pinned input, grammar and expected meaning
Extend	Independent grammar folders and a tested tutorial	A lexical entry or a construction family
Maintain	Builders, tests, guides and packaging checks	Review, releases and language-specific stewardship

The app and grammar tutorial work without a model subscription or the original developer's machine.

Next: extend families, then measure generalisation

1. **Reference and relatives.** Source a Greek *που* fragment; audit TTR identity and roles; specify supplement projection.
2. **Predicate reuse and ellipsis.** *John knows Mary. Bill does too.* Test antecedents and strict/sloppy readings.
3. **Nominals and events.** Coordination, possessives and plurality; then auxiliaries/voice with explicit event semantics.
4. **Community and evaluation.** Review analysis cards, tag a release, freeze fresh English/Greek samples and publish all outcomes.

Each extension needs positive examples, negative contrasts, meaningful output checks and reproducible evidence.

A short demonstration

Open DS Workbench / start in Parse

1. **Without AI** John, who Mary knows, walks.
Leave both switches OFF; follow Rules, LINK and MERGE.
2. **Construction assistance** John walks provided Mary walks.
Connect privately; LLM ON, Jev OFF. Inspect the relation and actual usage.
3. **Jev comparison** John lends a book to Mary.
LLM OFF, Jev ON; open *See Jev in action* and compare.
4. **Find the research** DS Library → search PCC or CL25.
Inspect review scope; return to Parse with the derivation preserved.

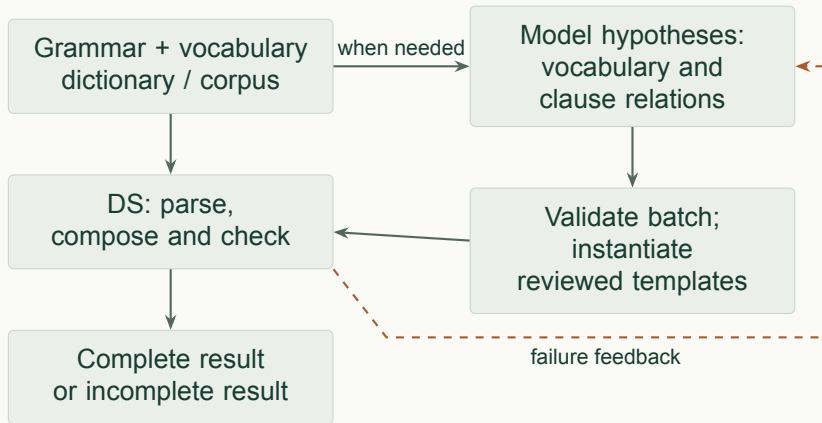
Appendix: DS–TTR and TTR-DS are different proposals

	DS–TTR content semantics	TTR-DS recasting
What TTR represents	Semantic records, fields and dependencies in DS processing	DS tree descriptions, parse states and action rules themselves
In this workbench	Inherited record-semantic machinery in a Python executor	Research card and prototype specification as future work
What follows	A separate TTR grammar must be tested on its own	The existing TTR dropdown is not an implementation of this recasting

A representation of DS in TTR does not require every semantic content to be a TTR record.

Cooper & Larsson (2025), §§1–2.2, 3, 4.8–5; DS Library record CL25 and linked analysis card.

Appendix: bounded proposal–compile–parse loop



At most three model calls, shared across a paragraph; 48-second helper budget. Provider failures preserve available grammar alternatives.

Appendix: the model never supplies this program

Structured model output

```
{"constructions": [{  
  "surface": "provided",  
  "relation": "condition",  
  "evidence": "Finite condition."  
}]}
```

Locally instantiated template

```
IF Ty(mltt:Prop)  
THEN causal-post(condition)  
ELSE abort
```

The atomic operation executes visible make/go/put steps, opens required content, and preserves local closure.

Historical operation names retain “causal”; the parameter selects the reviewed relation. Existing alternatives are retained.

Appendix: local content closure and Coq

A man walks if a dog walks.

$$\text{condition}(\Sigma x : \text{man.walk}(x), \quad \Sigma y : \text{dog.walk}(y))$$

Constructive declaration

```
Parameter condition : Type -> Type -> Prop.
```

- Each clause closes its own witnesses.
- Opaque relations accept content types in the supported exporter.
- Coq checks the exported representation; it does not prove the intended interpretation.

Appendix: implementation details that matter

Mechanism	Why it matters
Mutable-state isolation	Parser workers and branch-local freshening protect unrelated analyses.
Atomic validation	An invalid batch cannot leave a half-installed grammar.
Scope and replay	Restore context and metavariable state before retries; check meanings after revisions.
Generated grammar sections	Update builders and emitted files together; check idempotence.
Resource and search settings	Resolve installed assets; preserve lexical alternatives with <code>top_n=0</code> during coverage checks.

Code map: `workbench_api.py`, `assisted_parsing.py`, `construction_assistance.py`, `causal_effects.py`, `jev_comparison.py`.

Appendix: verification of the current workspace

8 October 2026 / application source 1aff525

Check	Observed result
Local targeted Python tests	81 passed: API, ASGI, rule playback and readings.
Focused browser regressions	6 rule-playback cases; 19 PCC cases as expected.
Workspace browser checks	7 grouped checks passed on source, staged bundle and public app, including desktop/mobile and AI permissions.
GitHub CI at this commit	Python 3.11/3.12 and installed-package jobs passed; Python 3.13 cancelled at the time limit.

These are overlapping software checks. They do not update linguistic coverage or model-quality estimates.

Appendix: foundations and further reading

- **Software.** Java DyLan, Dynamics of Language; Python dynamicsyntax.
- **Core DS.** Cann, Kempson & Marten (2005), *The Dynamics of Language*; Howes & Gibson (2021), *JoLLI* 30, 263–276.
- **Constructive semantics.** Chatzikyriakidis (2025), *Constructive Dynamic Syntax, Languages* 10(11):269.
- **Greek clitics.** Chatzikyriakidis (2010), thesis, chapters 3 and 6; Chatzikyriakidis (2012), *Lingua* 122(6), 642–672.
- **Project reports.** Complete technical walkthrough; theoretical-coverage catalogue; community contribution and maintenance guides.
- **Application and source.** [Public workbench](#); [community repository](#).

The slides describe the modified workbench snapshot. They do not claim complete implementations of the cited linguistic theories.

Appendix: the complete 18-category map

C01	Incremental growth and lexical programs	C10	Auxiliaries, restructuring, tense/voice
C02	Nominals, reference, quantification, scope	C11	Causal, temporal, conditional clauses
C03	Predication, valency and copulas	C12	Ellipsis and fragments
C04	Flexible order, case and discontinuity	C13	Questions, split turns, repair, grounding
C05	Relatives, resumption, extraction, islands	C14	Information structure and peripheries
C06	Adjectives, adverbs, PPs and comparison	C15	Agreement, plurality and typological tests
C07	Coordination and shared material	C16	Historical development and variation
C08	Anaphora and binding	C17	Learning, search and generation
C09	Clitic placement, clusters and person	C18	Semantic architectures and interfaces

Full catalogue: [docs/research/ds-theoretical-coverage.json](#) and companion report.

Appendix: extend an independent grammar

Copy the grammar; add a predicate and lexical entries; check meanings.

Semantic signature

`dance : human  $\rightarrow$  Prop`

Lexicon entries

`dances intransitive dance human`

`dance intransitive-base dance human`

Classical uses $e \rightarrow t$; both reuse the existing DS templates.

12 checks across both native backends; saved grammar hashes and expected meanings.

`examples/community/extend_grammar.py`; `docs/community/extending.md`.

Public Python API

```
from pathlib import Path
from dynamicsyntax import parse
result = parse("Mary dances.",
               Path("/path/to/my-grammar"),
               strict=True, top_n=0, trace=True)
print(result.semantics)
```

Appendix: public source and the next release

github.com/StergiosCha/DS_workbench

Available now

- Public source, issues and pull requests.
- UI, grammars, research library, slides and guides.
- Reproducible install and independent grammar tutorial.

Release work ahead

- Agree maintainers and review responsibilities.
- Resolve inherited licence and contributor terms.
- Tag a workbench release with artifacts and citation metadata.

Run from this source checkout; installing the upstream package by name does not reproduce the workbench.